```python
"""Run from this directory: python run_solution.py. Outputs are deterministic."""
from pathlib import Path
import json, time
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from scipy import signal
from common import FS, fixtures, read_wav, write_wav, relerr, snr, receive
from dsp_solution import analysis, synthesis, ola_filter, remove_jammer

def plot_tf(ax, x, L, hop, title):
    X, m = analysis(x, L, hop)
    f = np.fft.fftfreq(L, 1/FS)
    keep = f >= 0
    t = (np.arange(len(X))*hop-m['left']+(L-1)/2)/FS
    A = np.abs(X[:,keep]); A /= max(A.max(), 1e-15)
    ax.pcolormesh(t, f[keep], 20*np.log10(np.maximum(A.T, 1e-5)),
                  shading='auto', vmin=-70, vmax=0, cmap='magma')
    ax.set(xlim=(0,len(x)/FS), ylim=(0,4000), xlabel='Time (s)',
           ylabel='Frequency (Hz)', title=title)

def main():
    root = Path(__file__).resolve().parent
    out = root/'results'; out.mkdir(exist_ok=True)
    a = fixtures(root/'signals'); metrics = {}
    fig, axes = plt.subplots(3,1,figsize=(8,9),constrained_layout=True)
    for ax,L in zip(axes,[256,1024,4096]):
        plot_tf(ax, a['probe'], L, L//4, f'L={L}, duration={1000*L/FS:.0f} ms')
    fig.savefig(out/'uncertainty.png',dpi=160); plt.close(fig)
    tfrows=[]
    for L in [256,1024,4096]:
        H=L//4; X,m=analysis(a['transient'],L,H)
        energy=np.sum(np.abs(X)**2,axis=1)
        keep=energy>=.1*energy.max()
        tfrows.append(dict(L=L,window_ms=1000*L/FS,bin_hz=FS/L,hop_ms=1000*H/FS,
transient_10percent_energy_width_ms=1000*np.count_nonzero(keep)*H/FS))
    metrics['time_frequency']=tfrows
    fig,ax=plt.subplots(figsize=(8,4),constrained_layout=True)
    n=np.arange(1024); tone=np.cos(2*np.pi*1000*n/FS)+np.cos(2*np.pi*1080*n/FS)
    w=signal.get_window('hann',1024)
    for N in [1024,4096]:
        A=2*np.abs(np.fft.rfft(tone*w,N))/w.sum()
        ax.plot(np.fft.rfftfreq(N,1/FS),A,'.-',label=f'L=1024, N={N}')
    ax.set(xlim=(900,1200),xlabel='Frequency (Hz)',ylabel='Amplitude',title='Zero padding
samples the same windowed spectrum');ax.legend()
    fig.savefig(out/'zero_padding.png',dpi=160);plt.close(fig)

    fig, ax = plt.subplots(figsize=(8,4.5),constrained_layout=True)
    L=1024; N=65536; n=np.arange(L); window_rows=[]
    for name in ['boxcar','hann','hamming','blackman']:
        w=signal.get_window(name,L,fftbins=True)
        W=np.abs(np.fft.fft(w,N)); W/=W[0]
        minima=signal.find_peaks(-W[:N//2])[0]; first=int(minima[0])
        psl=20*np.log10(np.max(W[first:N//2]))
        window_rows.append(dict(window=name, null_to_null_hz=2*first*FS/N,
                                peak_sidelobe_db=float(psl),
                                enbw_hz=float(FS*np.sum(w*w)/np.sum(w)**2)))
        # Leakage experiment uses coherent-gain correction, not energy normalization.
        tone=.65*np.cos(2*np.pi*1003*n/FS)+.015*np.cos(2*np.pi*1140*n/FS)
        A=2*np.abs(np.fft.rfft(tone*w,N))/w.sum()
        ax.plot(np.fft.rfftfreq(N,1/FS),20*np.log10(np.maximum(A,1e-8)),label=name)

```

```
62      ax.set(xlim=(850,1350),ylim=(-90,5),xlabel='Frequency (Hz)',ylabel='Amplitude (dB re
    1)',title='Off-bin strong tone and weak neighbor'); ax.legend()
63      fig.savefig(out/'windows.png',dpi=160); plt.close(fig)
64      metrics['windows']=window_rows
65
66      rows=[]
67      for overlap in [0,.25,.5,.75,.875]:
68          H=int(1024*(1-overlap))
69          for win in (['boxcar','hann'] if overlap==0 else ['hann']):
70              start=time.perf_counter(); X,m=analysis(a['probe'],1024,H,win)
71              row=dict(overlap_percent=100*overlap,hop=H,window=win,frames=len(X))
72              try:
73                  y,d=synthesis(X,m)
74                  row.update(relative_error=relerr(a['probe'],y),
    min_denominator=float(d.min()),max_denominator=float(d.max()))
75                  assert row['relative_error']<1e-11
76              except ValueError as e:
77                  row['expected_failure']=str(e)
78                  assert win=='hann' and overlap==0
79              row['elapsed_seconds']=time.perf_counter()-start; rows.append(row)
80      metrics['overlap']=rows
81      h=signal.firwin(129,1200,fs=FS,window='hamming')
82      ola=ola_filter(a['mixture'],h,B=512)
83      ref=np.convolve(a['mixture'],h)
84      metrics['ola']=dict(relative_error=relerr(ref,ola), output_samples=len(ola),
85                          fft_length=1024,group_delay_samples=64)
86      assert relerr(ref,ola)<1e-11 and len(ola)==len(a['mixture'])+128
87      metrics['ola_block_sweep']=[]
88      for B in [128,512,2048]:
89          yy=ola_filter(a['mixture'],h,B)
90          metrics['ola_block_sweep'].append(dict(B=B,nfft=1<<(B+len(h)-2).bit_length(),
91                  blocks=int(np.ceil(len(a['mixture'])/B)),relative_error=relerr(ref,yy)))
92      # Independent edge-case checks, not only one friendly audio example.
93      rng=np.random.default_rng(52)
94      for N,B,M in [(1,1,1),(997,127,83),(1031,512,129)]:
95          x=rng.normal(size=N)+1j*rng.normal(size=N); hh=rng.normal(size=M)
96          assert relerr(np.convolve(x,hh),ola_filter(x,hh,B))<1e-11
97      write_wav(out/'ola_lowpass.wav',ola,scale=.8)
98
99      cleaned,diag=remove_jammer(a['mixture'])
100     write_wav(out/'input_mixture.wav',a['mixture'],scale=.8)
101     write_wav(out/'desired_reference.wav',a['melody'],scale=.8)
102     write_wav(out/'cleaned.wav',cleaned,scale=.8)
103     metrics['jammer']=dict(input_snr_db=snr(a['melody'],a['mixture']),
    output_snr_db=snr(a['melody'],cleaned))
104     # Apply the estimated mask unchanged to each source to separate attenuation from
    distortion.
105     def apply_gain(src,G,L,H):
106         X,m=analysis(src,L,H); return synthesis(X*G,m)[0]
107     jamrows=[]
108     for L in [256,1024,4096]:
109         for width in [30,60,120]:
110             y,dg=remove_jammer(a['mixture'],L,L//4,width)
111             jc=apply_gain(a['jammer'],dg['G'],L,L//4)
112             sc=apply_gain(a['melody'],dg['G'],L,L//4)
113             jamrows.append(dict(L=L,width_hz=width,output_snr_db=snr(a['melody'],y),
114
    jammer_attenuation_db=float(20*np.log10(np.linalg.norm(a['jammer'])/np.linalg.norm(jc))),
115                                 clean_relative_error=relerr(a['melody'],sc)))
116     metrics['jammer_sweep']=jamrows
117     # Equal-width stationary notch is a negative control.
118     X,m=analysis(a['mixture']); f=np.fft.fftfreq(1024,1/FS)
119     stationary=1-.97*np.exp(-.5*((np.abs(f)-2400)/60)**2)
120     ys,_=synthesis(X*stationary,m)
121
```

```
121     metrics['jammer']['stationary_notch_snr_db']=snr(a['melody'],ys)
122     fig,axes=plt.subplots(3,1,figsize=(8,9),constrained_layout=True)
123     for ax,x,title in zip(axes,[a['melody'],a['mixture'],cleaned],['Desired melody','Chirp
    jammer added','Estimated ridge suppression']):
124         plot_tf(ax,x,1024,256,title)
125     fig.savefig(out/'jammer.png',dpi=160); plt.close(fig)
126 
127     # Quantized WAV round trip is separate from float64 algorithm accuracy.
128     _,txwav=read_wav(root/'signals/qpsk_iq.wav',iq=True)
129     metrics['iq_wav_roundtrip_error']=relerr(a['tx'],txwav)
130     X,m=analysis(a['tx']); coherent,_=synthesis(X,m)
131     magnitude,_=synthesis(np.abs(X).astype(complex),m)
132     phase=rng.uniform(-np.pi,np.pi,len(X))
133     randomized,_=synthesis(X*np.exp(1j*phase[:,None]),m)
134 
    cases={'original':a['tx'],'coherent':coherent,'magnitude_only':magnitude,'frame_phase_reset'
    :randomized}
135     metrics['communications']={}
136     fig,axes=plt.subplots(1,3,figsize=(10,3.3),constrained_layout=True)
137     for name,x in cases.items():
138         row,z=receive(x,a['symbols'],a['bits'],a['pulse'])
139         row['waveform_relative_error']=relerr(a['tx'],x)
140         metrics['communications'][name]=row
141         if name!='original':
142             ax=axes[['coherent','magnitude_only','frame_phase_reset'].index(name)]
143             ax.scatter(z.real,z.imag,s=3,alpha=.4)
144 
    ax.set(xlim=(-1.3,1.3),ylim=(-1.3,1.3),xlabel='I',ylabel='Q',title=name.replace('_',' '));
    ax.set_aspect('equal'); ax.grid(alpha=.2)
145     fig.savefig(out/'constellations.png',dpi=160); plt.close(fig)
146     assert relerr(a['tx'],coherent)<1e-11
147     assert metrics['communications']['coherent']['ber']==0
148     metrics['checks']='PASS: identity, expected NOLA failure, real and complex full OLA,
    QPSK BER'
149     (out/'metrics.json').write_text(json.dumps(metrics,indent=2))
150     print(json.dumps(metrics,indent=2))
151     return metrics
152 
153 if __name__=='__main__':
154     main()
155 
```